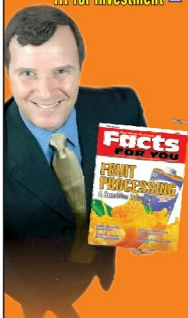


Grab
your copy
today

Look Before You Leap

... for investment



A business and economic affairs magazine that influences corporate decisions on the line of
Where should we put our money next?

Launched in 1980, Facts For You updates India's top decision-makers through its regular assortment of market surveys and other important industrial information.

For subscription & advertisements, contact:



EFY Enterprises Pvt Ltd

D-87/1, Okhla Industrial Area, Phase-I,
New Delhi - 110020

Phone: 011-26810601/02/03

Email: info@efyindia.com

Website: www.flymag.com

updated successfully, or none are. We will discuss DSU atomicity later.

Note that to prevent any threads from entering and executing any 'active function' while the DSU is going on, you may have to stall/delay the threads. Hence, it's important to reduce the time taken to apply the DSU, to avoid degrading the application response time.

DSU factors and issues

In our simple explanation of the DSU mechanism, we glossed over certain important issues, such as how to determine whether a code change is safe to be applied as a DSU; how to determine the application safe-points at which a DSU can be applied; and how to verify that a dynamically patched system is working correctly. These issues need to be addressed robustly for a DSU to be employed in production environments.

There are three important factors for DSUs to become practical in real-life environments:

- **Broad applicability:** It should be possible to apply a DSU to a broad set of applications and OS kernels, irrespective of whether single- or multi-threaded, whether written in C/C++ or Java, and whether a single or multiple functions/data structures are changed.
- **Minimal programmer burden:** DSUs should impose a minimum additional programming burden on the developer, should not cause major changes in the software development process, and should not impact performance.
- **Safety:** Programmers must be able to verify that a dynamically updated software application behaves as expected after the DSU is applied.

These properties are not easy to achieve in practice, making development of DSU systems quite difficult. Recall that the first step in the DSU is to determine code/data changes between old and new versions. In our simple example, the only change in 'foo' was 'strcpy' to 'strncpy'—a trivial

change that's internal to the function. However, this does not mean that the executable containing 'foo' only needs to be patched in one location. Why is this so?

The transformation of source code to object-level code is done by compilers, which perform a number of optimisations on the source code during the process. One of these optimisations is known as *inlining*, where a function's body is put 'in line' into calling code. In our example, if 'foo' was called from another function 'bar', it's possible that the compiler may have 'inlined' 'foo' into 'bar'. The same could possibly apply for more functions that call 'foo'. Hence, the DSU patch must ensure that all *inlinings* of 'foo' into its callers are also correctly updated.

Mere source-code comparison between old and new versions will not be able to detect changes in callers of 'foo', because to inline or not is a compiler 'decision'. So, a source-code comparison scheme would have to conservatively mark *all* callers of *foo* as modified. This can cause false positives in the 'active functions' list, increasing the time needed to apply the DSU.

Another major issue is ensuring that the application state is safe to apply the DSU. Our example used a simple criterion: none of the 'active functions' are currently active on the stack of any running thread. However, this check may not always be sufficient. Consider the following simple code snippet:

```
funcA()
{
    = lock(&l1);
}
FuncB()
{
    DoSomething();
}
FuncC()
{
    = unlock(&l1);
}
Main()
{
```